Banco de dados

Perambulando | Atualização: 18/10/2026

Base: `Data/Context/Context.cs`, `Data/Configurations`, migrations e `ContextModelSnapshot`. O esquema descrito é o esperado pelo código; a aplicação das migrations em um banco real não foi consultada.

1. Tecnologia e contexto

A API usa PostgreSQL por meio do Npgsql e EF Core. `Context` herda `IdentityDbContext<User>` e expõe `Categories`, `Hosts`, `Programmings` e `ProgrammingXCategories`. `OnModelCreating` aplica primeiro a configuração Identity e depois as configurações da assembly.

Item	Valor encontrado
Target da API	net10.0
EF Core	10.0.11
Provider Npgsql EF Core	10.0.3
Configuração da conexão	ConnectionStrings:DefaultConnection
Variável equivalente	ConnectionStrings__DefaultConnection

2. Tabelas de negócio

Tabela	Chave	Finalidade
Users	Id string/text	Conta Identity, dados básicos e flags de usuário
Hosts	Id integer identity	Estabelecimento pertencente ao usuário
Programmings	Id integer identity	Evento/programação pertencente ao host
Categories	Id integer identity	Cadastro de categorias
ProgrammingXCategories	Id integer identity	Vínculo de programação e categoria

Não há `Products` no snapshot atual. `SourceUrl` não faz parte de Programmings no modelo atual.

3. Relações e exclusões físicas

```
Users 1 -> N Hosts 1 -> N Programmings
Programmings 1 -> N ProgrammingXCategories N <- 1 Categories
```

Dependente	FK	Referência	Exclusão no snapshot
Hosts	UserId	Users.Id	Cascade
Programmings	HostId	Hosts.Id	Cascade
ProgrammingXCategories	ProgrammingId	Programmings.Id	Cascade
ProgrammingXCategories	CategoryId	Categories.Id	Cascade

As actions de exclusão de host e programação são lógicas: alteram IsActive. Essas operações não acionam cascade. A exclusão de usuário usa o Identity para remover fisicamente a conta; se o banco corresponder ao snapshot, o cascade pode alcançar hosts, programações e vínculos. Isso difere da desativação lógica.

4. Tabelas do Identity

Além de Users: `AspNetRoles`, `AspNetUserRoles`, `AspNetUserClaims`, `AspNetRoleClaims`, `AspNetUserLogins` e `AspNetUserTokens`. O EF usa também sua tabela de histórico de migrations.

Users persiste Id, UserName, NormalizedUserName, Email, NormalizedEmail, PasswordHash, Typeofuser, IsActive, isPro e CreatedAt.

`UserConfiguration` ignora no mapeamento: PhoneNumber, PhoneNumberConfirmed, AccessFailedCount, LockoutEnabled, LockoutEnd, TwoFactorEnabled, SecurityStamp, ConcurrencyStamp e EmailConfirmed. Portanto, a presença dessas propriedades herdadas em C# não significa que estejam persistidas. Em particular, não se deve presumir que bloqueio por tentativas e confirmação de e-mail estejam completos apenas por usar Identity.

5. Tipos e limites

Dados	Tipo PostgreSQL / limite
IDs inteiros e enums	integer
IDs Identity e FKs de usuário	text
Flags	boolean
Datas	timestamp with time zone
MinPrice e MaxPrice	real (float em C#)
Category.Name	varchar(100), não nulo
Hosts.Name / Neighborhood	varchar(150), não nulo
Hosts.Description	varchar(2000), não nulo
Hosts.Address	varchar(300), não nulo
Hosts.CNPJ / CEP / Phone	varchar(18) / varchar(9) / varchar(30), não nulos
Hosts.LogoUrl / InstagranUrl	varchar(2048), não nulos
UserName e e-mail normalizados ou originais	varchar(256)
Strings de Programming	text, não nulas no snapshot

Não nulo no banco não equivale a texto não vazio: strings vazias podem satisfazer a restrição NOT NULL. Os campos opcionais de URL são inicializados como string vazia, apesar de as colunas não aceitarem null.

Datas devem ser tratadas em UTC para esse mapeamento Npgsql. Preços são float/real atualmente, não decimal; essa é a representação existente.

6. Índices e integridade

- UserName normalizado possui índice único `UserNameIndex`.
- NormalizedEmail possui `EmailIndex` sem unicidade no snapshot. A exigência de e-mail único está na validação do Identity; não há garantia equivalente por índice único nessa coluna.
- Role normalizada possui índice único `RoleNameIndex`.
- Hosts.UserId e Programmings.HostId são indexados.
- Os IDs de programação e categoria na tabela de vínculo têm índices separados e FKs.
- Não há índice composto único de ProgrammingId + CategoryId. O create pode gerar vínculos repetidos, embora a leitura de categorias use Distinct e o edit rejeite IDs duplicados.
- Não há constraint de check no snapshot para preços, intervalo de datas, enums ou status ativo de categoria.

7. Persistência e transações

CreateWithCategoriesAsync e EditWithCategoriesAsync de ProgrammingRepository agrupam campos e vínculos antes de um único SaveChangesAsync. No provedor relacional, esse save usa transação por padrão quando necessário. Não há Unit of Work explícito.

As edições usam entidades rastreadas pelo contexto. Consultas de listagem e detalhes do host usam AsNoTracking em partes dos fluxos. Não há filtro global de IsActive no contexto: a disponibilidade é filtrada nos métodos dos repositórios.

Criar host e atribuir role ao usuário envolve etapas de persistência separadas; não há uma transação envolvendo o fluxo inteiro.

8. Migrations

Última migration encontrada: `20261016175528_AddProgrammingXCategory`, que cria a tabela de vínculos, PK, duas FKs em cascade e dois índices. Há migrations anteriores de criação Identity, renomeação de Users, categorias, hosts, programação e remoção de SourceUrl.

No startup de Development, a aplicação verifica se existem migrations e executa `Database.MigrateAsync`. Fora de Development não há aplicação automática nesse trecho do startup.

Comandos de referência, executados na pasta que contém o csproj:

```
dotnet ef migrations list
dotnet ef migrations add NomeDaMigration --output-dir Data/Migrations
dotnet ef database update
```

No Package Manager Console do Visual Studio, os equivalentes de criação/aplicação são `Add-Migration NomeDaMigration` e `Update-Database`, selecionando o projeto correto. Os comandos de criação/aplicação não foram executados nesta documentação.

Uma migration pode conter remoções e recriações, não apenas adições. Conferir Up, Down e o diff antes de aplicá-la faz parte da revisão da alteração.

9. Limites do esquema atual

Não há coordenadas, PostGIS, índice espacial ou consultas de proximidade nos arquivos consultados. Também não há tabela de refresh tokens ou revogação JWT implementada especificamente pela aplicação. A existência de AspNetUserTokens não representa um fluxo de refresh implementado.